Parallel Place-and-Route of FINN Dataflow Accelerators with a Static DFX Tile Overlay on the Alveo U55C*

Lukas Stasytis
TU Darmstadt

Report, 12 October 2026

Abstract

FINN compiles quantized neural networks into one streaming kernel per layer and hands the stitched design to Vivado for a single global place-and-route (P&R). On an Alveo U55C at 300 MHz this takes hours for finn-examples models and frequently fails timing. We describe a back end without a global step. The accelerator region of a cached shell is floorplanned once into a serpentine of clock-region tiles, each a DFX reconfigurable partition, joined by fixed, register-anchored link bundles. Per model, layers are synthesized out of context in parallel, packed onto tiles, and each tile is placed, routed and written as a partial bitstream independently. On a 64-core EPYC server, with 48 threads per build, VGG10 at 8× folding builds in 1278 s and meets 300 MHz timing in every tile, against 8619 s (timing failed) for FINN's Vivado flow and 5176 s for a baseline with the same parallel synthesis and shell but one global P&R; MobileNetV1 builds in 1541 s against 6222 s. The Vivado baselines at 300 MHz ran on 8 cores (Vivado's default thread cap is 8), which is a threat to these ratios. We also report an earlier RapidWright-stitched island flow (1.8–2.7× at 100 MHz, fails at 300 MHz), what was verified and what was not, the bugs that verification found, and the overlay's capacity limits.

1 Introduction

FINN [1] emits a dataflow graph (DFG) of hardware nodes (matrix-vector units, MVAU; thresholding; sliding-window generators; FIFOs; data-width converters; IODMAs) connected by AXI-Stream channels with ready/valid handshakes. The graph is latency-insensitive: only throughput matters, so any amount of pipelining may be inserted on a channel. The stock back end stitches all nodes into one IP, synthesizes the whole design and runs one global opt/place/phys_opt/route in Vivado. On the xcu55c this scales poorly: the larger designs measured here (300–400 k LUTs, up to 5000 DSPs) take 6200–10600 s at 300 MHz and three of five fail timing (Table 1). Vivado's placer and router by default use at most 8 threads, so a 64-core server is mostly idle.

The goal set for this work was an order-of-magnitude reduction of bitstream turnaround through parallel P&R of independent parts of the design, at the cost of some area efficiency, with functional correctness and timing at the target clock preserved. Cold builds are the normal case (folding, FIFO sizes or quantization change between iterations), so the gain must come from parallelism, not from reuse.

Contributions. (1) A *static tile overlay*: a cached DFX floorplan of the accelerator region with pre-routed, anchored links (Section 3.2). (2) A per-model flow that runs OOC synthesis, tile assignment and tile implementation in parallel and produces one partial bitstream per tile without any whole-design step. (3) Credit-based pipelined links whose FIFO depth is derived from the round-trip register count, with a simulation test. (4) Measurements of four flows on three finn-examples models at 100 and 300 MHz, a functional verification procedure, and an account of the bugs it caught and of the overlay's limits.

2 Background and related work

Partition-based parallel P&R. RapidStream 1.0 [2] splits a TAPA dataflow design into 32 islands on a U250, places one anchor register per inter-island net in thin anchor regions, implements the islands in parallel with Vivado, stitches with RapidWright [12] and routes the 5–10 % conflicting nets globally (Vivado or RWRoute [11]); 5–7× over Vivado. RapidStream 2.0 [3] removes the global route: islands become DFX reconfigurable partitions routed in abstract shells against pre-routed partition pins; 5.1–7.5× on a U280. RapidPnR [4] shows the ceiling of plain partition-and-merge without anchors (1.6–2.5×).

Page and overlay flows. PRflow [5] and DW [6] pre-build an interconnect (packet NoC, pipelined links through PR switchboxes) between fixed PR pages; PLD [7] nests 22 pages with abstract shells inside the Vitis shell (4.2–7.3× vs Vitis); HiPR [8] uses one-clock-region tiles per function; hierarchical pages [9] merge neighbours with nested DFX. Their shared lesson, which we follow: any post-hoc global route caps the speedup; pre-routed interfaces or reuse are needed for an order of magnitude.

Pre-implemented blocks. DynaRapid [10] places a library of pre-implemented relocatable components and routes with RWRoute (about 20× vs Vivado *Quick*), but only pays off with library reuse (library build about 10 h). Our earlier flow used it and dropped it for the cold-build reason above.

Position. The overlay is closest to RapidStream 2.0 and PLD: DFX tiles in abstract shells, fixed anchored links, no global route. Differences: FINN dataflow as the input; a serpentine grid sized to clock regions with static gaps; credit-based pipelined links with transit registers

*Code: FINN repository, branch `feature/rwislands`; log of all findings in `experiments/dynarapid/NOTES_ISLANDS.md`.

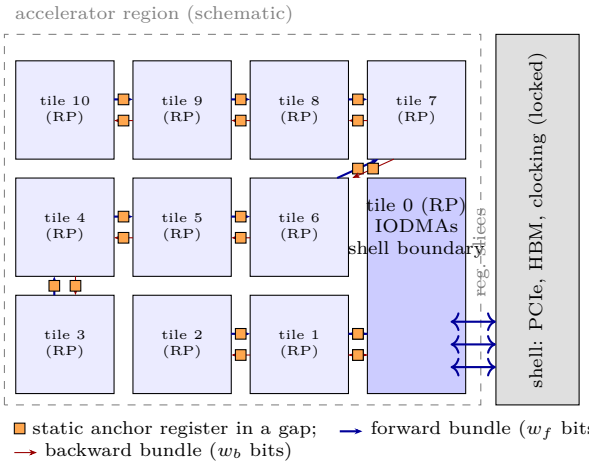


Figure 1: Static overlay (schematic, 11 of 22 tiles): tile RPs along the serpentine with anchored link bundles in the gaps; tile 0 absorbs the slot above it, so the 6→7 edge crosses diagonally.

through intermediate tiles; the static design cached per shell and clock; and Tcl pre-placement of the grey-box placeholders, without which grey-box detail placement takes tens of minutes and stalls beyond about 16 tiles.

3 System

3.1 Shell

The shell (PCIe XDMA, HBM/AXI interconnect, clocking) is implemented once per (board, clock, IODMA interface), locked and cached (`dynarapid/shell.py build_shell`). The accelerator (`finn_accel_core`) is a black-box cell inside it. Every shell↔core channel passes a two-entry register slice (`rwi_bridge_slice`) whose `ready` is driven directly by a register, so the shell side of every boundary path is a flop. All flows compared below use this shell, the same FINN front end and the same folding.

3.2 Static overlay

The overlay (`rwislands/overlay.py`, Fig. 1) is built once per (shell, tile grid, link widths) and cached under a key that also includes the overlay version and the Vivado release; we call the cached artefact the *static overlay* and the locked, routed design it contains the *static design*. The accelerator region beside the shell is cut into a serpentine (boustrophedon; "snake" in the code) of tiles aligned to clock regions. The production grid is 22 tiles of 2×2 clock regions (knobs `rows=120, crs=2`), tile 0 next to the shell being taller (`boundary_rows`) because it carries the IODMAs and all shell boundary signals. Each tile is a DFX reconfigurable partition (RP: a cell whose contents, the reconfigurable module, RM, are implemented separately and loaded by a partial bitstream; `HD.RECONFIGURABLE`, `SNAPPING_MODE ON`) with its own pblock. Pblocks are shrunk by one column and one or two rows on every side: the resulting *gaps* are static channels that hold the anchor registers and keep RP edges off clock-region boundaries (Vivado warns of limited routability there).

Links. Serpentine edge k (tile $k \leftrightarrow k{+}1$) carries a forward bundle of w_f bits and a backward bundle of w_b bits. Every bit passes exactly one static *anchor register* (RapidStream's term; DW's relay station) in the gap, so each RP boundary net has one static driver or load and one *partition pin* (the fixed routing node at which a net crosses the RP boundary), and every tile can be timed completely on its own. The production overlay uses $w_f = 2304$, $w_b = 32$ (VGG10 8× needs 2151/15 bits); a second 22-tile overlay with 1088/128 served the 100 MHz VGG10 0.25× runs. Because tile 0 abuts the shell, its boundary is timed against the bridges' register slices and needs no anchors.

Reset. The shell reset reaches the tiles through a static register chain along the serpentine (one register per edge) plus one `DONT_TOUCH` register per tile, again so that each RP boundary net has a single static driver (a 14.7 ns flat wire to far tiles failed setup, and synthesis merging equivalent registers violated the DFX rule Constraints 18-4430).

Grey boxes and the static build. The tiles are filled with *grey boxes* (a placeholder RM: one `DONT_TOUCH` LUT1 per port bit and one FDCE on the clock) so that the static design can be placed, routed and locked with real partition pins. Vivado's detail placer scales superlinearly on these tens of thousands of unconnected LUTs (29 min for one phase with 24 tiles); `PLACE_GREYBOX_TCL` pre-places every link's LUTs on the slices nearest the neighbouring tile before `place_design`, which pulls the partition pins to that edge (22 tiles: 5.5 min, 79 tiles: 17 min). The kernel clock net of the locked shell spans only the clock regions of its old loads; it is unrouted and re-routed under DFX rules. The static design is placed, routed, locked (`lock_design -level routing`), written as a full bitstream, and one *abstract shell* per tile (a checkpoint reduced to the static logic and routing a single RP needs for timing, `write_abstract_shell`) is written, 8 Vivado sessions in parallel. `write_bitstream` fails with more than 32 RPs in a design (Designutils 20-1700, Vivado 2024.2 and 2025.1), so the serpentine is cut at 32 tiles. One-off cost for 22 tiles on a shared 16-core lane: static 1050–1381 s plus abstract shells 677–1031 s; the shell itself about 950 s at 300 MHz. The static design closes timing at 100 and 300 MHz (300 MHz: worst negative slack WNS +0.29/+0.26 ns, worst hold slack WHS +0.009/+0.010 ns, 0 routing errors).

3.3 Per-model flow

`flow.py _overlay_build` runs the following steps. Steps 2 and 6 run in parallel; the planning steps 3–5 are serial but take seconds; nothing touches the whole design.

1. FINN front end output (folded, FIFO-sized dataflow ONNX) → `ZynqBuild`: IODMA insertion, partitions, HLS IP synthesis (one flat pool of spawned processes over all HLS nodes).
2. *Parallel OOC synthesis* of every distinct node (`synth_design`, single-threaded jobs, longest first; cheap FIFO/DWC/padding nodes batched in shared sessions; checkpoints cached per content hash). Optional *layer splitting* (`SplitLargeMVAU`) so that every part fits a tile (knob `part`: 45 % of a median tile by default).
3. *Tile assignment* (`plan_overlay`): nodes in DFG order along the serpentine; consecutive nodes share a tile while their synthesized utilization fits (LUT utilization tried 0.5–0.9; DSP/BRAM/URAM at `hard`

= 90–95 % of the tile's sites); the IODMAs go to tile 0. A node larger than every tile fails the plan with the reason.

4. *Link plan* (`link_plan`): every inter-tile stream gets a bit range on every edge it crosses (data+valid in the forward bundle, ready in the backward one); tiles in between pass it through. Bundle overflow fails the plan early.
5. *Tile netlists* (`tile_verilog`): a structural netlist per used tile, the member nodes plus one link transmitter per outgoing and one receiver per incoming stream and pass-through registers in transit tiles, linked with `link_design` from the OOC checkpoints (no global synthesis).
6. *Tile implementation in parallel* (`tile_tcl`): open the tile's abstract shell, `read_checkpoint -cell` the netlist into the RP, opt/place/phys_opt/route with the baseline's directives, reports, `write_bitstream -cell` (the partial bitstream), and the routed RM checkpoint. Threads per tile = lane threads / used tiles (2 on a 48-thread lane with 22 tiles).
7. Result: the cached static full bitstream plus one partial bitstream per used tile. No assembly step; for verification only, the static design and all routed RMs are read into one design to write a functional netlist.

3.4 Credit-based links

A link (`links.py`) is latency-insensitive with credit-style flow control. The transmitter registers data and valid through P_O output stages and the incoming ready through P_I stages; the receiver registers inputs through R_I stages into a FIFO and sends ready = "at most DEPTH − SLACK words buffered" through R_O stages. Since ready is late, the receiver must absorb every word in flight after it drops ready:

$$\text{in flight} = P_O{+}P_I{+}R_I{+}R_O + 2e + 2(e{-}1)\,T,$$

for a link over e edges (one anchor per edge and direction) with T registers per transit tile and direction. The FIFO depth is the smallest power of two ≥ 8 with depth − 3 $\geq$ in flight. The knob `pipe`= p sets $P_O{=}P_I{=}R_O{=}p$, $R_I{=}T{=}p{-}1$ (so $p{=}1$ gives $3 + 2e$); higher p buys timing margin at the cost of FIFO depth only, since inter-node latency is irrelevant in FINN dataflow. An xsim testbench over 1 and 3 edges with $p \in \{1, 2, 3\}$, random producer stalls and consumer back-pressure shows 0 lost, duplicated or reordered words at 1.000–1.008 cycles per word.

3.5 The earlier island flow

The island flow (`flow.py rw_islands_zynq_build`, `IslandStitcher.java`) cuts the DFG into islands, either a chain cut by min-max cost (*Islands (chain)*) or a packing onto 16-column × 60-row clock-region tiles (*Islands (grid)*), implements each island OOC in its own pblock with `CONTAIN_ROUTING` in parallel, with a two-entry register slice at both ends of every inter-island stream, stitches the routed islands into the shell with RapidWright (`populateBlackBox`, RWRoute partial routing of the inter-island nets with soft preserve), and finishes with one Vivado assembly: read the routed core into the shell, `route_design` on the remaining nets, hold fix, bitstream. The assembly is a global step. Islands (grid), VGG10 8×, 100 MHz: synthesis done at 352 s, islands at 1011 s, stitching at 1425 s, assembly at 2885 s of 2947 s, so stitching plus the global assembly is about half the build (read core about 430 s, route 1100–1400 s, bitstream about 160 s). At 300 MHz the assembly starts with 4051 unrouted nets, runs three rip-up-and-reroute iterations and ends at WNS −3.842 ns (21 894 failing endpoints): single-register crossings between islands plus a global final route do not close timing at high $f_{\max}$, which motivated the overlay.

4 Codebase and integration

The flow lives in `src/finn/util/rwislands/` (`overlay.py`: geometry, static Tcl, grey boxes, tile and link plans, tile netlists and Tcl; `links.py`: link RTL and FIFO sizing; `flow.py`: orchestration of all island-family flows; `floorplan.py`, `netlist.py`, `device.py`, `assembly.py`, `profiles.py`, `IslandStitcher.java` for the island flow) and reuses `dynarapid/shell.py` (shell, bridges), `components.py` (node synthesis, content-hash names, `FLOW_VERSION`) and `tools.py` (`run_vivado`: machine-wide slot locks, memory gating, timeouts). Integration is one option of FINN's existing transformation in `transformation/fpgadataflow/make_zynq_proj.py`: `ZynqBuild(..., dynarapid={"flow": "islands", "islands": "overlay"|"global"|"auto", "out_dir", "shell_lib", "workers"})`.
Overlay knobs come from the environment, e.g. `FINN_RWI_OVERLAY="cols=16,rows=120,crs=2, wf=2304,wb=32,gap_rows=2,hard=95,pipe=2"`;
grid and link widths are part of the overlay's cache key; `hard`, `part`, `pipe` and `tile0` are per-model knobs. Caches: shell per (board, clock, IODMA interface); overlay per (shell, grid, link widths, overlay version, Vivado version); node checkpoints per content hash and `FLOW_VERSION`; link modules per width and depth.

Experiment scripts (`experiments/dynarapid/`): `run_bitfile_experiment.py` (one build, `--mode vivado|global|islands|overlay|overlay_fast`), `run_vshell_timing.sh` (timed builds, `runs.jsonl`), `run_campaign.sh` (job queue over CPU lanes with disjoint cores, per-lane Vivado slots, memory watchdog), `build_overlay.py` (static overlay alone), `verify_accel.py` (functional check). Reproducing a point: build the shell and overlay once with `build_overlay.py`, then `run_bitfile_experiment.py --mode overlay --clk 3.333` on the FINN dataflow ONNX with `--shell-lib` pointing at the cached shells; every result directory contains `result.json` with status, per-tile timing and stamps.

5 Methodology

Machine. AMD EPYC 9554P, 64 cores / 128 threads, 755 GB RAM, Vivado 2024.2. The machine is split into *lanes* (disjoint sets of whole CCDs with their SMT siblings), one timed job per lane: lanes A and B have 24 cores / 48 threads each and ran all parallel flows; lane C (16 cores) is split into two 8-core / 16-thread sub-lanes for the Vivado baselines, since Vivado uses at most 8

Table 1: U55C, 300 MHz: wall time in s, WNS in ns (shell and overlay cached).

model	Vivado (FINN's flow)	Global	Islands (grid)	Overlay (`pipe=2`)	speedup vs Vivado
VGG10 0.25×	2511, +0.297	–	–	858, ≥+0.354 (3 tiles)	2.9×
VGG10 8×	8619, −0.516 fail	5176, +0.227	3204, −3.842 fail	1278, ≥+0.38 (22 tiles)	6.7× (4.0× vs Global)
MobileNetV1	6222, 0.000	4305, −0.033 fail	3491, −5.408 fail	1541, ≥+0.129 (20 tiles)	4.0×
VGG10 16×	9820, −0.188 fail[a]	–	–	does not fit[b]	–
ResNet50	10595, −1.729 fail	–	–	does not fit (12288-bit stream)[c]	–

[a] wrapper process killed by mistake; Vivado's implementation finished; time = job start to bitstream completion.
[b] 4994 DSP, greedy packing. [c] a StreamingFIFO on a 12288-bit stream exceeds every tile and the 2304-bit bundles (Section 8).

Table 2: U55C, 100 MHz: wall time in s; all runs met timing.

			Islands		
model	Vivado	Global	chain	grid	Overlay
VGG10 0.25×	2116	1168	918	–	816 (2.6×)
VGG10 8×	6742	4351	3058	2947	1189 (5.7×)
MobileNetV1	5287	3472	2940	2639	not run

threads by default. *Caveat:* the 300 MHz Vivado baselines ran on 8-core sub-lanes while the 100 MHz ones ran on 48-thread lanes; a 48-thread rerun of the 300 MHz VGG10 8× baseline was still running at the time of writing.

What is timed. Wall clock of the whole bitfile build from the FINN dataflow model: HLS, synthesis, P&R, bitstream. Shell and static overlay are cached and their one-off cost is reported in Section 3.2; runs that had to build the shell or overlay were discarded and rerun with warm caches. One run per point; earlier repeats showed about ±5–10 % variance.

Flows. *Vivado* (FINN's flow): FINN's own `ZynqBuild` with the same shell block design and Vivado's default directives (as Vitis v++): HLS, stitched IP, project synthesis, one global opt/place/phys_opt/route, bitstream. *Global*: our parallel per-node OOC synthesis and the cached shell, then one global P&R of the core inside the shell; it isolates the gain of parallel P&R from that of parallel synthesis and the cached shell. *Islands (chain)* and *Islands (grid)*: Section 3.5. *Overlay*: Section 3.2 with `pipe=2` at 300 MHz.

Models and metric. finn-examples on the U55C: VGG10 RadioML at 0.25× folding with residual adds ("0.25×"), 8× and 16×; MobileNetV1 (hand folding); ResNet50. Synthesized sizes: VGG10 8× about 310 k LUT and 4386 DSP (49 % of the device's DSPs); VGG10 16× 404 k LUT, 4994 DSP; MobileNet about 364 k LUT (estimate) and about 1150 BRAM. "Met" means 0 routing errors, WNS ≥ 0 and WHS ≥ 0; for Overlay every tile and the static design must meet. Clocks: 100 MHz (10 ns) and 300 MHz (3.333 ns).

6 Results

300 MHz (Table 1). Overlay is the only flow that meets timing on both VGG10 8× and MobileNetV1; Global meets it on VGG10 8× and Vivado on MobileNet (WNS exactly 0.000 ns). Speedups over Vivado are 2.9× (VGG10 0.25×), 6.7× (VGG10 8×) and 4.0× (MobileNet); over Global, 4.0× on VGG10 8×. Global itself is 1.7× faster than Vivado on VGG10 8×, which bounds what parallel synthesis and the cached shell contribute; the remaining factor comes from parallel tile P&R. Islands (grid) is faster than Global but fails timing by 3.8–5.4 ns because of its global final route. VGG10 16× exceeds the overlay's usable DSP capacity and ResNet50 has a stream too wide for a tile; both remain Vivado-only, and Vivado fails timing on both.

100 MHz (Table 2). Overlay is 2.6× and 5.7× faster than Vivado (VGG10 8× tiles at WNS ≥ +2.14 ns); the island flows reach 1.8–2.3× and Global 1.5–1.8×. `overlay_fast` (Vivado's RuntimeOptimized directives) on VGG10 8× took 1230 s, no gain over 1189 s: per-tile placement costs about 350 s either way.

Where the time goes. VGG10 8× at 300 MHz: the synthesis phase ends at 357 s, tiles take 449–846 s each and all are done at 1203 s; MobileNet: synthesis 586 s, tiles 317–760 s, done at 1346 s; VGG10 0.25×: synthesis 284 s, tiles 282–509 s. The synthesis phase is throughput bound (MobileNet: 16.3 k slot-seconds of single-threaded jobs on 32 Vivado slots; the longest node 160–180 s). A typical tile (25–30 k LUT, 256 DSP, 2 threads, 100 MHz) spends: link 40 s, open abstract shell 10 s, read RM 25 s, opt 45 s, place 350–450 s (device and netlist model about 70 s, global placement about 100 s, detail about 60 s, post-placement about 90 s), route 120–230 s, partial bitstream about 50 s. The fixed per-tile costs (about 600 s) plus the longest tile set the floor of the observed 800–1300 s.

Earlier island-flow results (2026-10-07, 100 MHz, previous FINN version, verified models): ZCU104 TFC 644→323 s, CNV 891→393 s, VGG10 2112→1205 s; U55C TFC 1528→722 s, VGG10 2980→1397 s, MobileNet (U250 folding) 5424→1989 s: 2.0–2.7×, the ceiling of partition-and-merge with a global final route.

7 Correctness and reliability

Verification method. `verify_accel.py` simulates in xsim the post-route functional netlist of the core (for Overlay: the static design with all routed RMs read in) together with the shell's bridge RTL, against FINN's stitched-IP RTL of the same graph, under one SystemVerilog testbench that programs AXI-Lite like the FINN driver and models the AXI memories. Inputs are random with per-frame amplitude scaling (`--vary-amplitude`) so that frames fall into different classes; the output buffers must be identical. A suspicious reference was cross-checked against FINN's Python execution of the streamlined ONNX model.

Verified: VGG10 0.25× Overlay at 100 MHz (8 frames covering 3 classes) and at 300 MHz with `pipe=2` (8 frames, same classes); VGG10 0.25× Islands (4 frames); VGG10 8× Overlay at 100 MHz (2 frames, 2 classes; 3.5 h of netlist simulation); earlier TFC (16 frames) and VGG10 on the ZCU104 (8 frames). **Not verified:** the MobileNet Overlay builds, any 300 MHz VGG10 8× build, and on-board execution: partial reconfiguration was not exercised on a board, only the assembled netlist was simulated.

Bugs found by this process (all fixed, with regression guards): (1) Thresholding units synthesized with all-zero threshold memories after a FINN upstream merge: the HDL reads `.dat` files by a relative path that OOC synthesis did not find (only a critical warning, Synth 8-4445); all non-Vivado results of two days were functionally wrong while their timings barely changed after the fix. Init files are now copied into the synthesis directory, Synth 8-4445 fails the node, and `FLOW_VERSION` was bumped. (2) Multi-edge links lost words with a FIFO slack sized for one edge; reproduced in xsim, fixed by the round-trip formula of Section 3.4. (3) Reset setup failures and merged reset registers (Section 3.2). (4) The static `place_design` stall and (5) the 32-RP bitstream limit (Section 3.2). (6) FINN's own baseline failed for ResNet50 (`ip_repo_paths` read back truncated); fixed. (7) Earlier, in the island flow: SLR-crossing islands, a dropped ElementwiseAdd input, a combinational bridge ready, partition cycles in the ResNet50 floorplan.

Status gating. Every flow result records a status; `ok` requires complete routing and setup and hold met in every tile, and failures are reported as results (no silent fallback in experiments). Cache keys include the flow version, the Vivado version and the knobs, so a changed flow never reuses stale artefacts.

8 Limitations and threats to validity

- *Capacity.* Tiles cover about 74 % of the device's DSPs (6720 of 9024; shell area and gaps are lost), and greedy in-order packing wastes up to about 30 % in DSP-heavy tiles (two 128-DSP parts in a 368-DSP tile), so VGG10 16× (4994 DSP) does not fit. Vivado can use the whole device.
- *Very wide streams.* ResNet50 has a StreamingFIFO on a 12288-bit stream (depth 2, about 12.3 k LUTs as LUTRAM) that is larger than every tile under the planner's limits, and the stream is far wider than the 2304-bit bundles: such streams must stay inside one tile, which needs a width converter or a different folding.
- *Floor.* The 32-tile cap bounds parallelism; per-tile fixed costs bound the build time at the observed 800–1300 s (Results).
- *One-off cost.* 35–40 min per (shell, clock, grid); a new IODMA interface means a new shell and overlay.
- *Fixed bundles.* A model whose streams exceed w_f/w_b fails the plan early.
- *Measurement.* Single runs; 300 MHz Vivado baselines on 8 cores vs 48 threads for the parallel flows (Section 5); the 48-thread rerun was not finished.
- *Verification coverage.* MobileNet and 300 MHz VGG10 8× Overlay builds unverified; no on-board test; the partial-reconfiguration deployment path (programming the static full bitstream and the partials on a U55C) not exercised.

9 Future work

TODO experiments: a ladder of longer Vivado jobs (ResNet50 on the overlay once its 12288-bit stream is narrowed or kept inside a tile, its 100 MHz Vivado baseline is running; VGG10 16× after capacity fixes); the 48-thread rerun of the 300 MHz VGG10 8× Vivado baseline (running); whole-machine runs (128 threads, about 5 threads per tile) to measure scaling; functional verification of the MobileNet and 300 MHz VGG10 8× Overlay builds; an on-board test with partial bitstreams; the parked ZCU104 jobs with the current flow. Missing pieces: capacity (smaller split parts, `part=30`; compute nodes in tile 0, implemented as knob `tile0`; tighter gaps; tiles over unused shell area; non-greedy packing); overlapping synthesis with tile P&R (start a tile when its members are synthesized) and synthesis on all lane threads (48 instead of 32 slots); reuse of unchanged tiles' partial bitstreams across builds, which DFX makes possible; and, for the island flow, credit-based pipelined links instead of single-register crossings, or retiring it in favour of the overlay.

References

[1] Y. Umuroglu et al., FINN: a framework for fast, scalable binarized neural network inference, FPGA 2017.

[2] L. Guo et al., RapidStream: parallel physical implementation of FPGA HLS designs, FPGA 2022.

[3] L. Guo et al., RapidStream 2.0: parallel implementation of latency-insensitive FPGA designs with nested DFX, ACM TRETS 2023.

[4] RapidPnR: netlist-level partitioning into a grid of islands, parallel P&R and merge, FCCM 2025 (poster).

[5] D. Park, Y. Xiao, N. Magnezi, A. DeHon, PRflow: fast FPGA compilation using partial reconfiguration, FPL 2018.

[6] Y. Xiao, A. Ahmed, A. DeHon, DW: fast linking of separately compiled FPGA blocks without a NoC, FPT 2020.

[7] PLD: fast FPGA compilation with nested DFX pages inside the Vitis shell, ASPLOS 2022.

[8] HiPR: high-level partial reconfiguration for fast incremental FPGA compilation, FPL 2022 / ACM TRETS 2023.

[9] Hierarchical PR pages with nested DFX, FPT 2022.

[10] DynaRapid: pre-implemented relocatable component library with RapidWright placement and RWRoute, FPL 2024.

[11] RWRoute: an open-source timing-driven router for commercial FPGAs, FPT 2021.

[12] C. Lavin, A. Kaviani, RapidWright: enabling custom crafted implementations for FPGAs, FCCM 2018.

[13] AMD, Vivado Design Suite User Guide: Dynamic Function eXchange (UG909); abstract shells.